🎸 Empório da Música — AI Customer Service Agent

Project developed for the **AI Engineer Technical Challenge at Artefact**. It consists of an intelligent virtual customer service agent for **Empório da Música**, a specialized musical instruments store located in Campo Grande, Mato Grosso do Sul (Brazil).

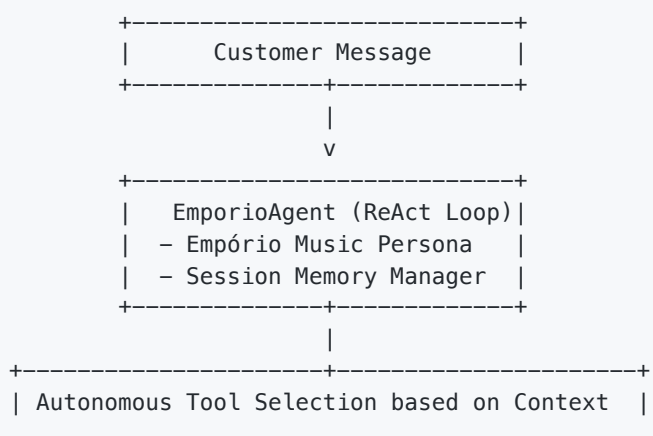
The agent handles customer inquiries, checks real-time inventory and catalog pricing, tracks customer orders, applies store exchange and return policies (Brazilian Consumer Protection Code - CDC), maintains strict factual accuracy, and handles out-of-scope inquiries (such as politely redirecting customers asking for guitar strings, cables, or accessories).

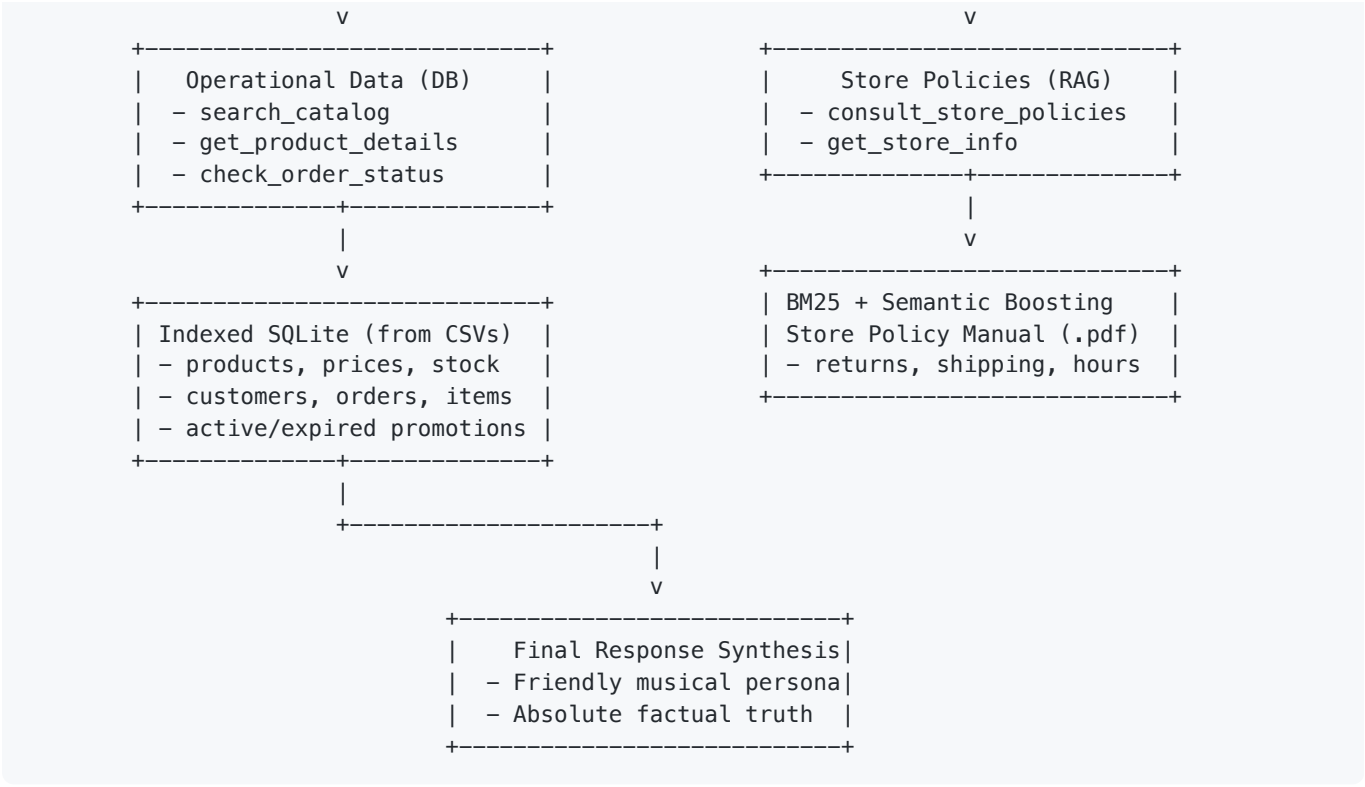
📋 Table of Contents

- Architecture Overview
- Repository Structure
- Installation and Environment Setup
- How to Run
  - Command Line Interface (CLI)
  - Web Interface (Streamlit)
  - Automated Tests
- Technical Decisions & Justifications
- Guardrails & Prompt Injection Defense
- Known Limitations & Future Roadmap
- Use of AI Coding Assistants

🏛️ Architecture Overview

The solution adopts a **Hybrid Architecture (Function Calling / ReAct + RAG + Relational Database)**:





1. **Operational Data Layer (SQLite)**: Operational CSV datasets (`products.csv` , `orders.csv` , `order_items.csv` , `customers.csv` , `categories.csv` , `promotions.csv`) are ingested into an indexed SQLite relational database on startup. Numeric constraints (e.g., acoustic guitars up to R$ 1,000.00) and stock availability use structured SQL queries typed with Pydantic models, guaranteeing **0% hallucination** on inventory numbers and prices.
2. **Policy Retrieval Layer (RAG)**: The store manual (`políticas_da_loja.pdf`) is extracted and partitioned into authoritative semantic sections, indexed using **BM25Okapi** with keyword normalization and metadata tag boosting (returns, warranty, business hours, installment plans, local shipping).
3. **Agent & Tool Orchestrator**: Identifies customer intent and orchestrates tool execution, enforcing store identity guidelines (e.g., active transparency on expired promotions and courteous redirection when asked for accessories).

📁 Repository Structure

The project follows modern Python community standards with package management powered by `uv` :

```
.
├── .env.example                    # Environment variables template
├── .gitignore                      # Ignores venvs, cache, and local secrets
├── pyproject.toml                  # PEP 621 package metadata & dependencies
├── README.md                       # Full technical documentation (English)
├── README.pdf                      # Converted publication-ready PDF document
├── data/                           # Datasets provided by the challenge
│   ├── desafio_tecnico_ai_eng - *.csv
│   └── políticas_da_loja.pdf
```

```
├── examples/                        # Real verified conversation transcripts (.md)
│   ├── 01_catalog_budget_search.md
│   ├── 02_store_info_and_hours.md
│   ├── 03_price_and_specs_check.md
│   ├── 04_refund_policy_non_trivial.md
│   └── 05_out_of_scope_redirection.md
├── src/
│   └── emporio_agent/
│       ├── __init__.py
│       ├── config.py                # Typed settings using Pydantic Settings
│       ├── cli.py                   # Interactive Rich terminal CLI
│       ├── ui.py                    # Streamlit web chat interface
│       ├── db/
│       │   ├── schema.py            # Pydantic schemas (Product, Order, Promo)
│       │   └── database.py          # SQLite loader & typed queries
│       ├── rag/
│       │   ├── parser.py            # PDF parser & semantic section chunker
│       │   └── retriever.py         # BM25 retriever with tag boosting
│       ├── tools/
│       │   └── definitions.py       # Function calling registry & JSON schemas
│       └── agent/
│           ├── prompts.py           # Empório da Música system prompt & persona
│           ├── memory.py            # Conversation turn history
│           ├── guardrails.py        # Pre-flight injection filter & post-flight sanitizer
│           ├── llm_client.py        # Multi-provider client (Gemini, Groq, Ollama, Mock)
│           └── core.py              # Autonomous agent loop
└── tests/
    ├── test_database.py             # Database query & filter unit tests
    ├── test_rag.py                  # Policy manual retrieval unit tests
    └── test_agent.py                # End-to-end agent scenario & guardrail tests
```

⚙️ Installation and Environment Setup

The project uses `uv`, the fast, modern Python package manager written in Rust.

Prerequisites

- Python `>= 3.11`
- `uv` installed (`curl -LsSf https://astral.sh/uv/install.sh | sh` or `brew install uv`)

1. Clone and Sync Dependencies

```
# Automatically sets up the virtual environment (.venv) and all dependencies
uv sync --extra dev
```

2. Configure Environment Variables

Copy the template file:

```
cp .env.example .env
```

Open `.env` and select your preferred LLM provider:

Option A: Google Gemini (Recommended — Free Tier)

Google AI Studio offers free tier access for `gemini-1.5-flash`, `gemini-2.0-flash`, and `gemini-3.5-flash-lite`.

1. Generate your free API key at aistudio.google.com.
2. In `.env`:

```
LLM_PROVIDER=gemini
GEMINI_API_KEY=your_gemini_api_key_here
GEMINI_MODEL=gemini-3.5-flash-lite
```

Option B: Groq (Free Tier via API)

1. Get a free API key at console.groq.com.
2. In `.env`:

```
LLM_PROVIDER=groq
GROQ_API_KEY=your_groq_api_key_here
GROQ_MODEL=llama-3.3-70b-versatile
```

Option C: Ollama (100% Local & Free)

Run fully offline on your machine with zero API keys:

```
LLM_PROVIDER=ollama
OLLAMA_BASE_URL=http://localhost:11434
OLLAMA_MODEL=llama3.2
```

Option D: Deterministic Mock Mode (Zero Setup / Offline Evaluation)

If no API key is supplied, the agent automatically runs in `mock` mode. It executes all tools, computes real stock/prices from the SQLite database, and formulates responses in the store's persona without making external network calls.

🚀 How to Run

1. Command Line Interface (CLI)

An interactive terminal experience with Markdown rendering and real-time tool inspection:

```
uv run emporio-agent
```

Available CLI shortcuts:

- `/debug`: Toggle real-time inspection of tool calls made by the agent.
- `/reset`: Clear conversation memory.
- `/sair` or `exit`: Exit the session.

2. Web Interface (Streamlit)

A full-featured chat interface with a sidebar containing quick question shortcuts, system status, and collapsible tool inspection drawers:

```
uv run streamlit run src/emporio_agent/ui.py
```

Open in your browser: `http://localhost:8501` or `http://127.0.0.1:8501`.

3. Automated Tests

A suite of 17 automated tests covering database queries, RAG retrieval, agent scenarios, and prompt injection defense:

```
uv run pytest -v
```

🧠 Technical Decisions & Justifications

Technical Decision	Selected Approach	Justification
Agent Architecture	**Tool Calling / ReAct with Pydantic**	Customer service for musical instruments requires exact numbers for prices, stock quantities, and order tracking. Pure LLM prompting hallucinates numerical details. Structured Pydantic tool calling allows the model to autonomously determine *when* to query operational databases and *when* to retrieve store policy guidelines.
Operational Data Layer	**Indexed SQLite Relational DB**	Flat CSVs do not provide fast indexed filtering or relational joins (e.g., linking `order_items` with `products` and `promotions`). SQLite is lightweight, serverless, 100% reproducible across developer machines, and enables compound filtering (`price <= 1000`, `category = 'Violões'`, `stock > 0`).

Technical Decision	Selected Approach	Justification
Policy Retrieval (RAG)	**Semantic Section Chunking + BM25Okapi**	Store policy manuals are regulatory documents structured into 10 explicit sections (consumer protection laws, local delivery fees, warranty terms). Section-boundary chunking respects contextual coherence, and BM25 with tag boosting delivers lexical precision on numerical time windows ("7 days", "30 days", "90 days", "R$ 500,00"), outperforming dense embedding models that often confuse differing legal deadlines.
Model & Provider Agnosticism	**Multi-Provider Adapter (Gemini / Groq / Ollama / Mock)**	Fulfills the requirement for free models by supporting Google AI Studio's free Gemini tier, Groq's high-speed free tier, local Ollama inference, and a deterministic offline mock provider for automated CI/CD pipelines.
Persona & Brand Tone	**Friendly Musical Consultant with Strict Guardrails**	Brand identity calls for an assistant that feels like "a musician friend who knows instruments", avoiding robotic or stiff corporate phrasing. The persona actively exercises transparency on expired promotional campaigns and politely redirects accessory requests.

🛡️ Guardrails & Prompt Injection Defense

In real-world production environments, conversational customer service agents are exposed to adversarial attacks:

- **Jailbreak & Prompt Injection Attempts**: Inputs such as *"ignore all previous instructions"*, *"act as DAN"*, or *"pretend you have no rules"*.
- **Infrastructure & Model Leakage**: Questions asking *"what AI model do you use?"*, *"who trained you?"*, or requests to dump internal system prompts.

To mitigate these vulnerabilities, the agent implements a **Defense-in-Depth** architecture:

1. **Programmatic Pre-Flight & Post-Flight Guardrails (`src/emporio_agent/agent/guardrails.py`)**:
   - **Pre-Flight Filter**: Inspects incoming messages before sending them to the LLM. Adversarial prompt injection attacks and meta-inquiries about underlying models are intercepted deterministically at the perimeter, saving token costs and responding immediately with the store's persona.
   - **Post-Flight Sanitizer**: Inspects outgoing responses to guarantee that internal model identifiers (Google, Gemini, OpenAI, etc.) are never leaked.
   - **Standard Protected Response**:

     > *"Hello! Everything good? 🎸 I am the exclusive virtual assistant for Empório da Música! My sole focus is to help you discover great musical instruments, check our catalog, pricing, orders, and answer store policy questions. How can I assist your sound today?"*

2. **System Prompt Hardening (`src/emporio_agent/agent/prompts.py`)**:

- Explicit persona grounding forbidding out-of-scope tasks (such as writing non-store code, discussing politics, or acknowledging third-party software providers).

⚠️ Known Limitations & Future Roadmap

With additional development time, the following improvements would be prioritized:

1. **Hybrid Dense + Sparse Vector Search**:
   - Combine BM25 with dense semantic embeddings (e.g., `text-embedding-3-small` or `bge-small-en-v1.5`) alongside a cross-encoder re-ranker for colloquial customer phrasing.
2. **WhatsApp Business Cloud API Integration**:
   - Connect webhooks to WhatsApp Cloud API or Evolution API, adding voice message audio transcription via Whisper and automated instrument picture cards.
3. **Distributed Session State with Redis**:
   - Persist customer session memory and cart state across distributed worker nodes with TTL-based retention.
4. **Continuous Evaluation Pipeline (LLM-as-a-Judge)**:
   - Automated quality gates using Ragas or DeepEval measuring *faithfulness*, *context recall*, and *tone adherence*.

🤖 Use of AI Coding Assistants

In full alignment with the Artefact evaluation guidelines, this project was developed in pair-programming with the platform's AI engineering coding assistant (Antigravity):

- **Architectural Planning**: Decomposing requirements into atomic tasks with test-driven checkpoints.
- **Implementation & Data Ingestion**: Writing the SQLite ingestion pipeline, PDF extraction with `pypdf`, and Pydantic validation schemas.
- **Test-Driven Verification**: Writing and verifying the 17-test pytest suite across catalog queries, RAG retrieval, and adversarial guardrail resilience.
- **Transparency**: The assistant accelerated boilerplate scaffolding and test authoring, while domain logic, store policies, and architectural decisions were driven strictly by the challenge specifications.